



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No. : 3

Team No. : 23

Project Title : Linux Memory-Mapped File Management System

Team Members

Roll Number	Student Name	Signature
2520030474	Santosh kumar bal	
2520030530	Charan tej.k	
2520030451	Bhargav.g	

1. Abstract

*The **Linux Memory-Mapped File Management System** is a system programming project that demonstrates efficient file management using memory-mapped files in the Linux operating system. Traditional file handling uses system calls such as `read()` and `write()`, which may involve repeated data transfers between user space and kernel space. Memory mapping provides an efficient alternative by mapping a file directly into a process's virtual address space, allowing the program to access and modify file contents as if they were stored in memory.*

*The project focuses on important **Operating Systems and System Programming concepts**, including virtual memory, process address spaces, memory management, file handling, system calls, and memory*

synchronization. Linux system calls such as `mmap()`, `munmap()`, `msync()`, `open()`, `fstat()`, and `close()` are used to implement the system.

The proposed system allows users to create or open files, map them into memory, read and modify their contents, synchronize changes with the file, and safely release the mapped memory. It demonstrates how the Linux kernel manages memory pages and maintains consistency between memory and secondary storage.

The expected outcome is a functional Linux-based system that provides practical understanding of memory-mapped files and demonstrates how they can improve file-access performance and reduce unnecessary data copying.

2. Problem Statement

Traditional file management in operating systems uses system calls such as `read()` and `write()` to transfer data between files and user-space memory. When large files or frequently accessed data are involved, repeated system calls and data copying can increase processing overhead and reduce performance. There is a need for an efficient method of accessing and modifying file data while minimizing unnecessary data transfers.

The **Linux Memory-Mapped File Management System** aims to address this problem by using the Linux `mmap()` system call to map file contents directly into a process's virtual address space. This allows applications to access file data through normal memory operations instead of repeatedly performing file I/O operations.

The project is relevant to **Operating Systems and Systems Programming** because it demonstrates key concepts such as virtual memory, address-space management, memory mapping, file handling, system calls, page management, and synchronization between memory and storage. It also provides practical understanding of how user applications interact with the Linux kernel.

The proposed system will provide basic operations such as opening files, mapping files into memory, reading and modifying data, synchronizing changes, and safely unmapping memory. The project aims to demonstrate an efficient and practical approach to file management while providing hands-on experience with Linux system-level programming.

3. Objectives

1. **To design and develop** a Linux-based system for efficient file management using memory-mapped files.
2. **To implement and demonstrate** Linux system calls such as `mmap()`, `munmap()`, `msync()`, `open()`, `fstat()`, and `close()` for file and memory management.
3. **To understand and demonstrate** Operating System concepts such as virtual memory, process address spaces, memory mapping, page management, and synchronization.
4. **To evaluate the benefits** of memory-mapped file access by reducing unnecessary data copying and improving the efficiency of file read and write operations.

4. Proposed Methodology

The **Linux Memory-Mapped File Management System** will be designed and implemented as a Linux-based system programming application using the **C programming language** and standard Linux system calls. The system will use memory mapping to provide efficient access to file data through the process's virtual address space.

The implementation will begin by allowing the user to **create or open a file** using the `open()` system call. The file size and required properties will be obtained using `fstat()`. The `mmap()` system call will then be used to map the file into the process's virtual memory with appropriate read/write permissions.

Once the file is mapped, the application will allow users to **read, modify, and update file contents directly through memory operations**. Changes made to the mapped memory will be synchronized with the actual file using `msync()`. Proper error handling will be included to handle invalid files, mapping failures, and permission issues.

After completing the required operations, the system will use `munmap()` to release the mapped memory and `close()` to close the file descriptor.

The overall flow of the system will be:

User Input → Open/Create File → Obtain File Information → Map File into Memory → Read/Modify Data → Synchronize Changes → Unmap Memory → Close File

This methodology demonstrates the interaction between user applications, virtual memory, file systems, and the Linux kernel while providing an efficient approach to file management.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Memory Mapping (<code>mmap()</code>)	Maps a file directly into the process's virtual memory, allowing the project to access file contents as memory.
<code>munmap()</code>	Removes the memory mapping when the file is no longer required and releases the associated virtual memory region.
<code>open()</code> / <code>close()</code>	Opens the file for reading/writing and closes the file descriptor after operations are completed.
<code>msync()</code>	Synchronizes modified memory-mapped data with the underlying file on disk.
Virtual Memory Management	Demonstrates how the OS manages virtual address spaces and maps file-backed pages into memory.
File Descriptors & File I/O	Provides a mechanism to identify and manage the opened files used for memory mapping and file operations.

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520030474	Santosh kumar bal	Project design, implementation of memory mapping using mmap(), and overall integration
2520030530	Charan tej.k	File handling using open(), close(), and file descriptor management
2520030451	Bhargav.g	Implementation of msync() and synchronization of mapped files

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Provides the operating system environment for implementing and testing the project.
C / C++	Used to develop the memory-mapped file management system.
GCC Compiler	Compiles the C/C++ source code into executable programs.
Linux System Calls / POSIX APIs	Used for mmap(), munmap(), open(), close(), and msync() operations.
VS Code	Used for writing, editing, and managing the project source code.
Git / GitHub	Used for version control, code management, and collaboration.

8. Expected Outcome

*The project is expected to develop a functional **Linux Memory-Mapped File Management System** that demonstrates efficient file handling using memory mapping. The system should allow users to create, open, read, write, modify, and manage files through memory-mapped regions. It will provide better understanding of how Linux manages memory and files, while reducing unnecessary data copying and improving access efficiency.*

9. GitHub Repository

GitHub Repository Link:  [santoshxdev/-Linux-Memory-Mapped-File-Management-System_OSSP](https://github.com/santoshxdev/Linux-Memory-Mapped-File-Management-System_OSSP)

Repository Created: ☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature: